Python Assessment Project 4

City Information REST API — Python + Flask/FastAPI + SQLite

Assessment Objective

• Build a Python REST API for managing city and weather information.

• Demonstrate practical Python programming, REST API development, database operations, validation, error handling, and testing.

Business Scenario

• A company wants a lightweight service to store and retrieve city-level weather information.

• The application should expose REST endpoints that can be consumed by a web application or another data pipeline.

Technology Requirements

• Python 3.x

• Use either Flask or FastAPI.

• SQLite for persistence.

• Use JSON for API requests and responses.

• Use pytest for unit/API testing.

Data Model

```
city_id
city_name
state
country
temperature
humidity
weather_condition
recorded_at
```

Required API Endpoints

```
GET /cities
GET /cities/<id>
POST /cities
PUT /cities/<id>
DELETE /cities/<id>

GET /cities?state=Telangana
GET /cities?min_temperature=30

GET /statistics
```

POST Example

```
{
  "city_name": "Hyderabad",
  "state": "Telangana",
```

```
  "country": "India",
  "temperature": 32.5,
  "humidity": 65,
  "weather_condition": "Sunny"
}
```

Statistics Endpoint

```
{
  "total_cities": 10,
  "average_temperature": 31.5,
  "maximum_temperature": 38,
  "minimum_temperature": 24
}
```

Functional Requirements

• Create a new city/weather record.

• Retrieve all city records.

• Retrieve a city by ID.

• Update an existing city.

• Delete a city.

• Filter cities by state.

• Filter cities by minimum temperature.

• Return aggregated temperature statistics.

• Return appropriate HTTP status codes.

Validation Requirements

• city_name, state, and country must not be empty.

• temperature must be numeric.

• humidity should be between 0 and 100.

• weather_condition must be a valid non-empty value.

• Return meaningful error messages for invalid requests.

• Return 404 when a requested city does not exist.

Expected Project Structure

```
city_api/
  app.py
  models.py
  database.py
  routes.py
  validators.py
  requirements.txt
  tests/
    test_api.py
  README.md
```

Testing Requirements

• Test successful POST, GET, PUT, and DELETE operations.

• Test invalid input.

• Test missing records.

• Test filtering.

• Test the statistics endpoint.

• Aim for meaningful automated test coverage.

Assessment Criteria

Area	Marks
Python fundamentals	10
Flask/FastAPI	20
REST API design	15
SQLite/database	15
Validation and error handling	10
Unit/API testing	10
Code quality	10
Documentation	5
Problem solving	5
Total	100

Submission Requirements

• Submit the complete project source code.

• Include requirements.txt.

• Include the SQLite database or a database initialization script.

• Include automated tests.

• Include README.md with setup, execution, API examples, and testing instructions.

Bonus Challenges

• Add pagination to GET /cities.

• Add sorting by temperature or city name.

• Add Swagger/OpenAPI documentation if using FastAPI.

• Add logging.

• Add environment-based configuration.

• Containerize the API using Docker.

• Add a simple frontend or API client.

• Add a CI/CD workflow that runs the tests automatically.